
A4 WRITEUP

Problem 1: kmeans_omp

=====

kmeans_omp Timing Table

~~~~~

Fill in the following table on measuring the performance of your 'kmeans\_omp' program on Zaratan. Replace 00.00 entries with your actual run times. You can use the provided 'time-kmeans-omp-slurm.sh' which, after minor adjustments, should produce all times needed to fill it in. INCLUDE THE SLURM JOB OUTPUT FILE IN YOUR SUBMISSION; this will allow graders to verify your times if in doubt.

| -----     |                    |                    |                    |
|-----------|--------------------|--------------------|--------------------|
| Data File |                    |                    |                    |
| Procs     | digits_all_5e3.txt | digits_all_1e4.txt | digits_all_3e4.txt |
| -----     |                    |                    |                    |
| 1         | 2.79               | 16.95              | 43.53              |
| 4         | 1.01               | 4.83               | 12.70              |
| 8         | 0.72               | 2.85               | 7.43               |
| 16        | 0.62               | 1.90               | 4.94               |
| 36        | 0.53               | 1.44               | 3.63               |
| 64        | 0.52               | 1.40               | 3.24               |
| 85        | 0.53               | 1.49               | 3.15               |
| 128       | 0.61               | 1.60               | 3.27               |
| -----     |                    |                    |                    |

### kmeans\_omp Discussion Questions

~~~~~

Analyze your table of results and answer the following questions.

1. Did using more processors result in speedups?
2. Describe any trends or anomalies you see in the timings and speculate on their causes - e.g. was there a steady increase in runtimes, steady decrease, or jagged changes in timing?
3. Describe your general approach on how you used OpenMP to subdivide the work for the two main portions of the algorithm

- (A) How did you subdivide the Cluster Center Calculation Phase?

- (B) How did you subdivide the Data Assignment Phase?

In both cases mention the shared data involved and how access to it was coordinated to ensure correctness.

1. Yes. More processors gave clear speedups for all three datasets. For example, on digits_all_3e4.txt runtime dropped from 43.53s at 1 proc to 3.15s at 85 procs.
2. The trend is mostly decreasing runtime as processor count increases. The biggest improvements happen early from 1 to 16 to 36. After that, the gains are smaller, and at very high counts there are small fluctuations such as a slight slowdown at 128 versus 85. This is expected from parallel overhead costs when many threads are used.
3. I parallelized over data points. Each thread computed partial sums in a private local array for all clusters and dimensions, then those partial sums were reduced into shared cluster sums. After reduction, each cluster sum was divided by its count to get the new center. This avoids races on shared cluster-center values.
I then parallelized over data points again. Each thread computed nearest cluster assignments for its own points, while tracking local cluster counts and a local change count. These local totals were merged into shared clust.counts and global nchanges in a synchronized section, so updates remained correct.

Problem 2: kmeans_cuda

=====

kmeans_cuda Timing Table

~~~~~

Fill in the following table on measuring the performance of your 'kmeans\_cuda' program on Zaratan. Replace 00.00 entries with your actual run times. You can use the provided 'time-kmeans-cuda-slurm.sh' script which, after minor adjustments, should run the serial and CUDA codes on necessary data. INCLUDE THE SLURM JOB OUTPUT FILE IN YOUR SUBMISSION; this will allow graders to verify your times if in doubt.

The columns are for each of 3 data files that are provided and run in the job script.

| -----   |                    |                    |                    |
|---------|--------------------|--------------------|--------------------|
|         | Data File          |                    |                    |
| Program | digits_all_5e3.txt | digits_all_1e4.txt | digits_all_3e4.txt |

|               |      |       |       |
|---------------|------|-------|-------|
| kmeans_serial | 2.55 | 16.59 | 42.31 |
| kmeans_cuda   | 1.05 | 2.28  | 2.28  |

## kmeans\_cuda Discussion Questions

Analyze your table of results and answer the following questions.

1. Did using the GPU result in speedups?

Yes, and the speedup grows with data size:

5e3 : 2.55s / 1.05s = 2.4x

1e4 : 16.59s / 2.28s = 7.3x

3e4 : 42.31s / 2.28s = 18.6x

The serial runtime grows roughly linearly with `ndata` (and with the iteration count, which also grows on the larger files). The CUDA runtime is essentially flat between 1e4 and 3e4 (both 2.28s) because at these sizes the per-iteration kernel-launch and host->device memcopy overhead (copying `nchanges` and counts back each iteration) dominates over the compute work itself. Tripling the data is absorbed in the noise. This is a common characteristic of GPUs where small problems are not worth the transfer/launch overhead, but the speedup widens fast as the compute side scales.

2. Describe your general approach on how you used GPU threads/blocks to parallelize the algorithm.

The host loads the data once, flattens the (`ndata x dim`) feature matrix and the (`nclust x dim`) centers matrix to row-major 1D buffers, and ships the data, initial assignments, and initial counts to the GPU one time. After that, each iteration of the k-means loop is three consecutive kernel launches plus a small `cudaMemcpy` of `nchanges` and counts back to the host (so the host can print the iteration line and check for convergence). The data features array never moves between host and device after the initial copy. Centers and assignments live on the device for the whole loop and are only copied back at the end (for the PGM files and the confusion matrix).

Threads-per-block is fixed at 256. The number of blocks is chosen per-kernel based on which axis the kernel parallelizes over (see below).

#### (A) Cluster Center Calculation Phase:

We used the lecture's "1 thread per output cell" pattern from the matrix-multiply slides. There are  $n_{\text{clust}} \times \text{dim}$  output cells, so we launch  $\text{ceil}(n_{\text{clust}} \times \text{dim} / 256)$  blocks of 256 threads. Thread  $\text{tid}$  maps to cluster  $c = \text{tid} / \text{dim}$  and dimension  $d = \text{tid} \% \text{dim}$ . That thread loops over all  $n_{\text{data}}$  points and accumulates  $\text{data}[i \times \text{dim} + d]$  for every  $i$  where  $\text{assigns}[i] == c$ , then divides by  $\text{counts}[c]$  at the end.

There was no coordination. Each thread writes exactly one output cell and no other thread writes that cell, so there are no atomics, no critical sections, and no `__syncthreads()` needed. The "redundant computation" is that every thread re-reads the full `assigns[]` array which is the cost of avoiding atomics on a complex multi-thread reduction. With  $n_{\text{clust}} \times \text{dim} \approx 15,680$  threads on an A100 and  $n_{\text{data}}$  up to 30,000 inner-loop iterations per thread, the GPU's compute is well utilized.

Something to note: Because every thread sums the same subset of points in the same index order ( $i = 0..n_{\text{data}}-1$ ) as the serial code, the floating-point result is bit-identical to the serial output.

#### (B) Data Assignment Phase:

There is 1 thread per data point.  $\text{ceil}(n_{\text{data}} / 256)$  blocks of 256 threads are launched. Thread  $i$  computes the squared distance from  $\text{data}[i]$  to every cluster center using the same nested-loop structure as the serial code, finds the best cluster, writes its own `assigns[i]` (there is no race as each thread owns its own  $i$ ), and then `atomicAdd`s 1 to `counts[best_clust]` and `atomicAdd`s 1 to a global `nchanges` counter if its assignment flipped.

integer `atomicAdd` on `counts[]` (size  $n_{\text{clust}} \sim 20$ ) and on the single `nchanges` scalar. Integer `atomicAdd` is order-independent so the result is deterministic. The distance computation itself uses the same inner-loop order as serial, so distances are also bit-exact, which makes

tie-breaking identical to the serial implementation.

3. Describe the GPU kernels that you wrote and how they are used in the main computation to accomplish speedup.

- kernel\_compute\_centers(ndata, dim, nclust, d\_data, d\_assigns, d\_counts, d\_centers)

One thread per (cluster, dim) cell as described in the Cluster Center Calculation Phase of question 2.

Computes  $\sum_{i: \text{assigns}[i]=c} \text{data}[i*\text{dim} + d]$  and divides by counts[c]. This kernel both zeroes (sum starts at 0.0 locally) and accumulates and divides in a single pass -- the sum, divide, and zero-empty-cluster cases from the serial loop are fused into one kernel call.

- kernel\_zero\_ints(arr, n)

A tiny kernel that writes 0 to each entry. Used to reset counts[] each iteration before the assignment kernel runs. (nchanges is reset with cudaMemcpy, which is cheaper than a kernel launch for a single int.)

- kernel\_assign(ndata, dim, nclust, d\_data, d\_centers, d\_assigns, d\_counts, d\_nchanges)

One thread per data point. Distance computation, best-cluster pick, write-back of assigns[i], and atomicAdds on counts[best\_clust] and nchanges as described in the Data Assignment Phase of Question 2.

The speedup comes from both: (i) the center kernel's  $\text{nclust} * \text{dim} \sim 15,680$  independent threads each doing ndata-length sums in parallel replacing the serial version's double nested loop, and (ii) the assign kernel's ndata threads each doing their own  $\text{nclust} * \text{dim}$  distance computation replacing another serial double loop. The data array stays on the GPU the whole run, so the only host-device traffic per iteration is two small ints worth of state (nchanges + counts), which keeps the loop overhead bounded.

Problem 3: Optional kmeans\_blas

=====

kmeans\_blas Timing Table

~~~~~

Fill in the following table on measuring the performance of your 'kmeans_blas' program on Zaratan. Replace 00.00 entries with your actual run times. INCLUDE THE SLURM JOB OUTPUT FILE IN YOUR SUBMISSION; this will allow graders to verify your times if in doubt.

Program	Data File		
	digits_all_5e3.txt	digits_all_1e4.txt	digits_all_3e4.txt
kmeans_serial	02.70	16.81	43.88
kemans_blas	01.18	07.19	24.16

NOTE: Write your own Timing Script

There is NOT a provided timing script / SLURM job for this. It is suggested that you adapt the 'time-kmeans-cuda-slurm.sh' script for this to run the plain Serial and BLAS versions of kmeans on 3 datasets in that file. Make sure to modify the options concerning the need for GPUs (they are not needed for the BLAS runs)

```
,---
| ## SELECT PARTITION FOR GPUs
| #SBATCH --partition=gpu    # change to "standard"
| #SBATCH --gpus=a100:1     # eliminate option
| ---
```

kmeans_blas Discussion Questions

Analyze your table of results and answer the following questions.

1. Did using the BLAS result in speedups? Describe what you see in the Table.
 - a. Using BLAS did result in speedups across all test cases. For digits_all_5e3.txt, the runtime improved from 2.70 seconds to 1.18 seconds, giving a speedup of about 2.29x. For digits_all_1e4.txt, the runtime improved from 16.81 seconds to 7.19 seconds, giving a speedup of about 2.34x. For digits_all_3e4.txt, the runtime improved from 43.88 seconds to 24.16 seconds, giving a speedup of about 1.82x. As we can see, speedup was most impactful for 5e3 and 1e4 data sets. It may not be as impact for 3e4 due to additional overhead from vector BLAS calls.

2. Describe which section of the serial code you used BLAS to address the main computation sections of the algorithm. Indicate which BLAS functions that you used for these two:
- (A) BLAS functions for the Cluster Center Calculation Phase

In the cluster center calculation phase, I used BLAS when accumulating data points into their assigned cluster centers. In the serial version, this phase uses an inner loop over all dimensions to add each feature of a data point into the corresponding feature of the cluster sum. In the BLAS version, I replaced that feature by feature addition with `cblas_daxpy`.

For each data point i , the code finds its assigned cluster c and performs `clust->features[c] += data->features[i]` using `cblas_daxpy`. After all data points have been added into the cluster sums, the code divides each cluster center by its count to compute the average. That final division step is still done with a regular loop.

- (B) BLAS functions for the Data Assignment Phase

Here, we used BLAS to compute the squared Euclidean distance between each data point and each cluster center. The serial version manually subtracts feature values and accumulates the sum of squared differences. In the BLAS version, this work is split into BLAS vector operations. `cblas_dcopy` copies the current data point's feature vector into a temporary vector called, `cblas_daxpy` subtracts the cluster center from that temporary vector, and `cblas_ddot` computes the dot product of `diffs` with itself resulting in the squared Euclidean distance. The code repeats this for each cluster and chooses the cluster with the smallest squared distance as the new assignment for that data point.

Work Disclosure
=====

(A) *Human Collaborators:* Aside from the 1-2 people submitting this assignment, the following people contributed ideas and discussion to the completion of this work INCLUDING course staff members. (Write NONE if no collaborators were involved.)

NONE

(B) *Resource Utilization:* The following resources such as websites, course notes, artificial intelligence tools (LLMs/ChatBots/etc.) were utilized in the completion of this work. Include course materials such as textbooks and lecture slides as well. (Write NONE if no resources were used [which would be hard to believe]).

- 1) We referred to the lecture slides (Shared Memory Architectures, Threads and OpenMP, Open Multi-Processing (OpenMP), GPU Architecture and CUDA Programming) for all coding and write up parts
- 2) We utilized ChatGPT model 5.4 to ask clarifying questions about
 - a) Viable blas functions for speeding up the serial code regarding loops and vector functions
Example prompt: What does cblas_daxpy do and provide an example of its usage relative to serial c code.
- 3) We utilized ChatGPT model 5.4 about Zaratan file upload and download commands
 - a) Example prompt: what is the command to upload a folder to zaratan. Answer: scp -r "folder path" <username>@login.zaratan.umd.edu

(C) *Ownership of Work*: I / We affirm that all parties listed below have contributed to each solution presented in this document. All parties are capable of describing how the solutions were derived, how they apply to the problem, and that they were created in accordance with the course's PRIME DIRECTIVE.

Signed,

Edward Song, Eric Chen, Daniel Wang